

PcapHunt

PcapHunt is a complete, production-quality CLI tool for CTF players and network-forensics analysts. It hunts through `.pcap` / `.pcapng` files packet-by-packet and automatically extracts useful human-readable or encoded data — dramatically reducing the need to manually inspect thousands of packets in Wireshark.

Why PcapHunt?

During CTFs and incident-response investigations, PCAP files often contain thousands of packets. Manually scrolling through them in Wireshark is time-consuming and error-prone. PcapHunt performs a **rapid first-pass triage**, extracting:

- plaintext strings
- Base64, hex, and URL-encoded data
- URLs, domains, IP addresses, emails
- credentials and tokens
- CTF flags
- hashes and JWTs
- embedded file signatures (PNG, PDF, ZIP, etc.)
- suspicious high-entropy blobs

It associates every finding with specific packet numbers, reconstructs TCP streams in deep mode, and deduplicates repeated content so you get clean, actionable output.

Features

- **14 modular detectors** covering the most common CTF/forensics data types
- **TCP stream reassembly** (`--deep`) to catch flags split across packets
- **Recursive decoding** (Base64 → hex → plaintext, etc.)
- **Deduplication** prevents hundreds of identical files
- **Rich terminal UI** with progress bars and color-coded results
- **JSON output** (`--json`) for programmatic consumption
- **Search filtering** (`--search`) to narrow results
- **Configurable** via `~/.config/pcaphunt/config.toml`
- **Graceful error handling** — malformed packets never crash the tool
- **Fast** — works incrementally without loading the entire PCAP into memory

Installation

From source (recommended for development)

```
git clone <repo-url>
cd pcaphunt
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

System install with pipx

```
pipx install pcaphunt
```

Requirements

- Python 3.11+
- `scapy` (PCAP parsing)
- `rich` (terminal UI)

Usage

Basic scan

```
pcaphunt capture.pcap
```

Deep mode (TCP stream reassembly)

```
pcaphunt capture.pcap --deep
```

Specify output directory

```
pcaphunt capture.pcap -o ./results
```

JSON output to stdout

```
pcaphunt capture.pcap --json
```

Search for a specific string

```
pcaphunt capture.pcap --search "flag"
```

Full CLI reference

```
$ pcaphunt --help
usage: pcaphunt [-h] [-o OUTPUT] [--deep] [--min-length MIN_LENGTH]
               [--no-dedup] [--json] [--quiet] [--search SEARCH] [--version]
               [pcap]

PcapHunt - Hunt for useful data in PCAP/PCAPNG files

positional arguments:
  pcap                Path to PCAP/PCAPNG file

options:
  -h, --help          show this help message and exit
  -o, --output OUTPUT  Output directory (default: ./pcaphunt_output)
  --deep              Enable deep mode (TCP stream reassembly, etc.)
  --min-length MIN_LENGTH
                     Minimum plaintext string length (default: 6)
  --no-dedup          Disable deduplication
  --json              Output findings as JSON to stdout
  --quiet             Suppress terminal output
  --search SEARCH     Filter findings by search string
  --version           Show version and exit
```

Also runnable without installation:

```
python -m pcaphunt capture.pcap
```

Examples

Example 1: Basic scan

```
$ pcaphunt challenge.pcap

[ _ \ / _ ] [ _ _ _ ] [ | | | ] [ _ _ _ _ ] [ | | ]
[ | _ ) ] [ | | | ] [ / _ \ / _ ] [ | | | ] [ | | | ] [ _ \ | ] [ _ ]
[ | _ / ] [ | | ] [ ( | \ \ \ _ ] [ | | | ] [ | | | ] [ | | | ]
[ | _ \ \ \ \ _ ] [ | _ / ] [ | | ] [ \ _ ] [ | | ] [ | \ \ ]

PCAP Content Hunter

[*] Input: challenge.pcap
[*] Packets: 18,421

[+] Analyzing packets... 100%

PCAPHUNT RESULTS

Plaintext      1,284
Base64         37
Hex            19
Url Encoded    11
Urls           43
Ip Addresses   112
Domains        28
Emails         6
Credentials    4
Flags          2
Hashes         13
Jwt            1
Files          7
Suspicious     8

🚩 FLAGS FOUND:
flag{hidden_in_stream}
CTF{reconstructed_flag}

🔑 Credentials detected: 4
(saved to output/credentials/)

[+] Results: ./pcaphunt_output/
[+] Analysis completed in 4.82 seconds
```

Example 2: Deep mode

```
$ pcaphunt challenge.pcap --deep -o ./deep_results
```

Deep mode enables TCP stream reassembly, allowing PcapHunt to detect strings split across multiple packets.

Example 3: JSON output

```
$ pcaphunt challenge.pcap --json --quiet | jq '.[] | select(.type == "flags")'

{
  "type": "flags",
  "packet_numbers": [100, 101],
  "protocol": "TCP",
  "source": "10.0.0.5:43122",
  "destination": "10.0.0.10:80",
  "offset": 42,
  "original": "CTF{this_is_a_flag}",
  "decoded": "CTF{this_is_a_flag}",
  "confidence": 1.0,
  "fingerprint": "..."}
}
```

Output Structure

Default output directory: `./pcaphunt_output/`

```
pcaphunt_output/
├─ plaintext/
├─ base64/
├─ hex/
├─ url_encoded/
├─ urls/
├─ ip_addresses/
├─ domains/
├─ emails/
├─ credentials/
├─ flags/
├─ hashes/
├─ jwt/
├─ files/
├─ suspicious/
├─ streams/
├─ summary.txt
└─ findings.json
```

Each finding is saved as a human-readable `.txt` file:

```
PcapHunt Finding
=====

Type: Base64
Packet: 81
Protocol: TCP
Source: 10.0.0.5:43122
Destination: 10.0.0.10:80
Offset: 42
Confidence: 0.98

Original:
SGVsbG8gQ1RG

Decoded:
Hello CTF
```

If multiple findings exist in one packet, files are numbered:

```
packet_12_01.txt
packet_12_02.txt
```

Detector Explanations

Detector	What it finds	Notes
<code>plaintext</code>	Printable ASCII/UTF-8 strings	Configurable minimum length
<code>base64</code>	Valid Base64 with meaningful decoded content	Uses padding, alphabet, and content heuristics
<code>hex</code>	Hexadecimal strings with printable decoded output	Validates even length and printable result
<code>url_encoded</code>	Percent-encoded sequences	Supports recursive decoding
<code>urls</code>	<code>http://</code> , <code>https://</code> , <code>ftp://</code> , etc.	Extracts full URLs
<code>ip_addresses</code>	IPv4 and IPv6 addresses	Validates with Python's <code>ipaddress</code> module
<code>domains</code>	Domain names	Filters fake TLDs and noise
<code>emails</code>	Email addresses	Simple structure validation
<code>credentials</code>	<code>password=</code> , <code>token=</code> , <code>api_key=</code> , etc.	Values saved to <code>credentials/</code> only
<code>flags</code>	<code>flag{...}</code> , <code>CTF{...}</code> , <code>HTB{...}</code> , etc.	Configurable regex patterns
<code>hashes</code>	MD5, SHA1, SHA256, SHA512	Uses length and context heuristics
<code>jwt</code>	JSON Web Tokens	Decodes header and payload when valid
<code>files</code>	File magic bytes (PNG, PDF, ZIP, etc.)	Reports offset and type
<code>suspicious</code>	High-entropy data (compressed/encrypted)	Entropy threshold ≥ 7.5

Deep Mode

Deep mode (`--deep`) enables:

- **TCP stream reassembly** — concatenates payloads from the same TCP stream to find strings split across packets
- **Additional decoding passes** — more aggressive recursive decoding
- **Deeper file signature analysis** — scans more aggressively for embedded files
- **Entropy analysis** — runs on all substantial payloads

Deep mode is slower but essential when flags or secrets are fragmented across multiple packets.

Example: split flag

Packet 100: `CTF{this_is_` Packet 101: `a_flag}`

Without `--deep` : no flag found. With `--deep` : `CTF{this_is_a_flag}` detected and associated with packets 100–101.

Configuration

Create `~/.config/pcaphunt/config.toml`:

```
# PcapHunt Configuration File
# Place this file at ~/.config/pcaphunt/config.toml

# Minimum string length for plaintext extraction
min_length = 6

# List of enabled detectors
enabled_detectors = [
    "plaintext",
    "base64",
    "hex",
    "url_encoded",
    "urls",
    "ip_addresses",
    "domains",
    "emails",
    "credentials",
    "flags",
    "hashes",
    "jwt",
    "files",
    "suspicious",
]

# Custom flag regex patterns
flag_patterns = [
    'flag\\\[^\]+\\\'',
    'FLAG\\\[^\]+\\\'',
    'CTF\\\[^\]+\\\'',
    'ctf\\\[^\]+\\\'',
    'ICT\\\[^\]+\\\'',
    'HTB\\\[^\]+\\\'',
    'picoCTF\\\[^\]+\\\'',
]

# Default output directory
output_directory = "./pcaphunt_output"

# Maximum recursive decode depth
max_decode_depth = 3

# Enable deduplication by default
deduplication = true

# Enable deep mode by default
deep_mode_default = false
```

Development

```
# Install in editable mode with dev dependencies
pip install -e ".[dev]"

# Run tests
pytest tests/ -v

# Run specific test
pytest tests/test_integration.py -v
```

Testing

PcapHunt includes a comprehensive pytest suite with synthetic PCAP fixtures. Tests cover:

- plaintext, Base64, hex, URL-encoded extraction
- URL, IP, domain, email detection
- flag, hash, JWT, file-signature detection
- entropy calculation
- deduplication logic
- recursive decoding
- malformed packet handling
- TCP stream reconstruction (including split-flag detection)
- output generation
- CLI argument parsing

All tests use small synthetic PCAPs created in-memory with Scapy — no external files required.

Limitations

- **Protocol parsing:** Uses Scapy heuristics; exotic or heavily fragmented protocols may not be perfectly parsed
- **TCP reassembly:** Simple concatenation (no out-of-order or gap handling). Works well for typical CTF traffic but not a full TCP stack
- **File extraction:** Detects signatures and reports offsets but does not always extract complete files when data is truncated
- **Encrypted traffic:** TLS payloads are opaque unless decrypted with keys (not supported)
- **Performance:** Very large PCAPs (>1 GB) will take longer; incremental reading keeps memory usage reasonable

License

MIT License — see [LICENSE](#).

Contributing

Contributions welcome! Areas of interest:

- Additional detectors (e.g., QR codes, steganography hints)
- Better protocol parsers (SMB, HTTP/2, QUIC)
- GUI or web viewer for results
- Performance optimizations for massive PCAPs